



Experiment No.: 3

Title: Mutation Testing



Batch: B-1

Roll No.: 16010422234

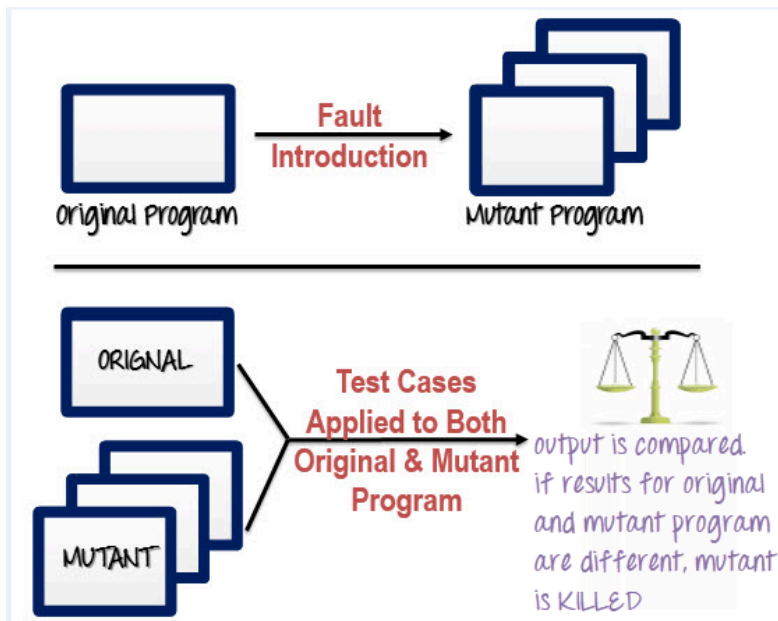
Experiment No.: 3

Aim: To perform mutation testing for measuring the test case coverage.

Resources needed: Eclipse IDE, PIT Mutation Testing.

Theory:

Mutation Testing is a type of software testing where we mutate (change) certain statements in the source code and check if the test cases are able to find the errors. It is a type of White Box Testing which is mainly used for Unit Testing.

**Mutation Testing Types:**

- **Value Mutations:** An attempt to change the values to detect errors in the programs. We usually change one value to a much larger value or one value to a much smaller value. The most common strategy is to change the constants.
- **Decision Mutations:** The decisions/conditions are changed to check for the design errors. Typically, one changes the arithmetic operators to locate the defects and also we can consider mutating all relational operators and logical operators (AND, OR , NOT)
- **Statement Mutations:** Changes done to the statements by deleting or duplicating the line which might arise when a developer is copy pasting the code from somewhere else.

How to Create Mutant Programs?

A mutation is nothing but a single syntactic change that is made to the program statement. Each mutant program should differ from the original program by one mutation.

Some of sample mutation operators:

- GOTO label replacement
- Return statement replacement
- Statement deletion
- Unary operator insertion (Like - and ++)
- Logical connector replacement
- Comparable array name replacement
- Removing of else part in the if-else statement
- Adding or replacement of operators
- Statement replacement by changing the data
- Data Modification for the variables
- Modification of data types in the program

Example 1:

Original Program	Mutant Program
If (x>y)	If(x<y)
Print "Hello"	Print "Hello"
Else	Else
Print "Hi"	Print "Hi"



Example 2:

Example- If(x>y) replace x and y values If(5>y) replace x by constant 5	Example- If(x==y) We can replace == into >= and have mutant program as If(x>=y) and inserting ++ in the statement If(x==++y)
--	---

How to execute mutation testing?

Following are the steps to execute mutation testing:

Step 1: Faults are introduced into the source code of the program by creating many versions called mutants. Each mutant should contain a single fault, and the goal is to cause the mutant version to fail which demonstrates the effectiveness of the test cases.

Step 2: Test cases are applied to the original program and also to the mutant program. A test case should be adequate, and it is tweaked to detect faults in a program.

Step 3: Compare the results of the original and mutant program.

Step 4: If the original program and mutant programs generate the different output, then that mutant is killed by the test case. Hence the test case is good enough to detect the change between the original and the mutant program.

Step 5: If the original program and mutant program generate the same output, Mutant is kept alive. In such cases, more effective test cases need to be created that kill all mutants.

Automation of Mutation Testing:

Mutation testing is extremely time consuming and complicated to execute manually. To speed up the process, it is advisable to go for automation tools. Automation tools reduce cost of testing as well.

List of tools available -

Ninja Turtles- .net mutation testing tool

Mutagenesis- PHP mutation testing framework

Jester- Mutation testing tool for Java

PITest- Mutation testing tool for Java and the JVM

Mutation Score:

The mutation score is defined as the percentage of killed mutants with the total number of mutants.

- $\text{Mutation Score} = (\text{Killed Mutants} / \text{Total number of Mutants}) * 100$

Test cases are mutation adequate if the score is 100%. Experimental results have shown that mutation testing is an effective approach for measuring the adequacy of the test cases. But, the main drawback is the high cost of generating the mutants and executing each test case against that mutant program.

Procedure:

Step 1: To download PITest- Mutation Testing[Eclipse-> help-> Market places -> type "PITest"-> select PITest from the list-> next]

Step 2: write the test cases and run [right click on the test case file-> run as-> PITest]

Step 3: Observe the result and write additional test cases to achieve a 100% score.

Results: (Document printout as per the format)

Write test cases and perform PITest- Mutation Testing for 100% score and document the same.

src/main/java/com/bankapp/Account.java

```
package com.bankapp;

/**
 * Represents a single bank account.
 * This class handles the core data and logic for an account,
 * such as its number, owner, and balance.
 */
public class Account {
    // --- Attributes ---
    private String accountNumber;
    private String accountHolderName;
    private double balance;

    // --- Constructor ---
    public Account(String accountNumber, String accountHolderName,
double initialDeposit) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = initialDeposit;
    }

    // --- Getters (to access private data safely) ---
    public String getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolderName() {
        return accountHolderName;
    }

    public double getBalance() {
        return balance;
    }

    // --- Core Functionality (Methods) ---

    /**
```

```

    * Adds funds to the account.
    * @param amount The amount to deposit. Must be positive.
    * @return true if the deposit was successful, false otherwise.
    */
    public boolean deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
            return true;
        } else {
            System.err.println("Deposit amount must be positive.");
            return false;
        }
    }

    /**
     * Withdraws funds from the account.
     * @param amount The amount to withdraw. Must be positive and not
exceed the balance.
     * @return true if the withdrawal was successful, false otherwise.
     */
    public boolean withdraw(double amount) {
        if (amount <= 0) {
            System.err.println("Withdrawal amount must be positive.");
            return false;
        }
        if (amount > this.balance) {
            System.err.println("Insufficient funds. Current balance: "
+ this.balance);
            return false;
        }
        this.balance -= amount;
        return true;
    }

    @Override
    public String toString() {
        return String.format("Account [Number: %s, Holder: %s, Balance:
$%.2f]",
            accountNumber, accountHolderName, balance);
    }
}

```

src\main\java\com\bankapp\Bank.java

```
package com.bankapp;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
import java.util.UUID;

/**
 * Manages all bank accounts and provides a command-line interface
 * for users to interact with the banking system.
 */
public class Bank {
    // Using a Map to store accounts, with the account number as the
    // key for fast lookups.
    private Map<String, Account> accounts = new HashMap<>();
    private Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.run(); // Start the application
    }

    /**
     * The main application loop that displays the menu and handles
     * user input.
     */
    public void run() {
        System.out.println("Welcome to the Simple Banking App!");
        while (true) {
            printMenu();
            System.out.print("Enter your choice: ");
            String choice = scanner.nextLine();

            switch (choice) {
                case "1":
                    createAccount();
                    break;
                case "2":
                    depositToAccount();
                    break;
                case "3":
```

```

        withdrawFromAccount();
        break;
    case "4":
        checkBalance();
        break;
    case "5":
        System.out.println("Thank you for using the Simple
Banking App. Goodbye!");
        return; // Exit the loop and the program
    default:
        System.err.println("Invalid choice. Please enter a
number between 1 and 5.");
    }
    System.out.println(); // Add a blank line for readability
}

private void printMenu() {
    System.out.println("-----");
    System.out.println("1. Create New Account");
    System.out.println("2. Deposit Funds");
    System.out.println("3. Withdraw Funds");
    System.out.println("4. Check Balance");
    System.out.println("5. Exit");
    System.out.println("-----");
}

private void createAccount() {
    System.out.print("Enter account holder's name: ");
    String name = scanner.nextLine();

    // Generate a simple unique account number
    String accountNumber = "ACC-" +
UUID.randomUUID().toString().substring(0, 8).toUpperCase();

    Account newAccount = new Account(accountNumber, name, 0.0);
    accounts.put(accountNumber, newAccount);

    System.out.println("✅ Account created successfully!");
    System.out.println("Your new account number is: " +
accountNumber);
}

```



```
private Account findAccount() {
    System.out.print("Enter account number: ");
    String accountNumber = scanner.nextLine();
    Account account = accounts.get(accountNumber);
    if (account == null) {
        System.err.println("❌ Account not found.");
    }
    return account;
}

private void depositToAccount() {
    Account account = findAccount();
    if (account != null) {
        try {
            System.out.print("Enter amount to deposit: ");
            double amount = Double.parseDouble(scanner.nextLine());
            if (account.deposit(amount)) {
                System.out.printf("✅ Deposit successful. New
balance: $%.2f%n", account.getBalance());
            }
        } catch (NumberFormatException e) {
            System.err.println("❌ Invalid amount. Please enter a
number.");
        }
    }
}

private void withdrawFromAccount() {
    Account account = findAccount();
    if (account != null) {
        try {
            System.out.print("Enter amount to withdraw: ");
            double amount = Double.parseDouble(scanner.nextLine());
            if (account.withdraw(amount)) {
                System.out.printf("✅ Withdrawal successful. New
balance: $%.2f%n", account.getBalance());
            }
        } catch (NumberFormatException e) {
            System.err.println("❌ Invalid amount. Please enter a
number.");
        }
    }
}
```

```

    }

    }

    private void checkBalance() {
        Account account = findAccount();
        if (account != null) {
            System.out.println("Account Details:");
            System.out.println(account); // Uses the overridden
toString() method in Account
        }
    }
}
}

```

src\test\java\com\bankapp\AccountTest.java

```

package com.bankapp;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class AccountTest {

    private Account account;

    @BeforeEach
    void setUp() {
        // Create a fresh account with a starting balance for each test
        account = new Account("ACC-123", "John Doe", 500.00);
    }

    // --- Original "Happy Path" Tests ---
    @Test
    void testSuccessfulDeposit() {
        assertTrue(account.deposit(100.00));
        assertEquals(600.00, account.getBalance());
    }

    @Test
    void testSuccessfulWithdrawal() {
        assertTrue(account.withdraw(200.00));
        assertEquals(300.00, account.getBalance());
    }
}

```

```
}

// --- ADDITIONAL TESTS TO KILL ALL MUTANTS ---

@Test
void testDepositWithNegativeAmount() {
    assertFalse(account.deposit(-50.00));
    assertEquals(500.00, account.getBalance());
}

@Test
void testDepositZeroAmount() {
    assertFalse(account.deposit(0));
    assertEquals(500.00, account.getBalance());
}

@Test
void testWithdrawWithNegativeAmount() {
    assertFalse(account.withdraw(-50.00));
    assertEquals(500.00, account.getBalance());
}

@Test
void testWithdrawInsufficientFunds() {
    assertFalse(account.withdraw(1000.00));
    assertEquals(500.00, account.getBalance());
}

@Test
void testWithdrawExactBalance() {
    assertTrue(account.withdraw(500.00));
    assertEquals(0.00, account.getBalance());
}

@Test
void testWithdrawZeroAmount() {
    assertFalse(account.withdraw(0));
    assertEquals(500.00, account.getBalance());
}
}
```

```

- Mutators
=====
> org.pitest.mutationtest.engine.gregor.mutators.ConditionalBoundaryMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
> org.pitest.mutationtest.engine.gregor.mutators.returns.PrimitiveReturnsMutator
>> Generated 1 Killed 1 (100%)
> KILLED 1 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
> org.pitest.mutationtest.engine.gregor.mutators.VoidMethodCallMutator
>> Generated 3 Killed 0 (0%)
> KILLED 0 SURVIVED 3 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
> org.pitest.mutationtest.engine.gregor.mutators.returns.BooleanTrueReturnValsMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
> org.pitest.mutationtest.engine.gregor.mutators.MathMutator
>> Generated 2 Killed 2 (100%)
> KILLED 2 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====

```

```

> org.pitest.mutationtest.engine.gregor.mutators.returns.BooleanFalseReturnValsMutator
>> Generated 2 Killed 2 (100%)
> KILLED 2 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
> org.pitest.mutationtest.engine.gregor.mutators.returns.EmptyObjectReturnValsMutator
>> Generated 3 Killed 0 (0%)
> KILLED 0 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 3
=====
> org.pitest.mutationtest.engine.gregor.mutators.NegateConditionalsMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
- Timings
=====
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : 2 seconds
> build mutation tests : < 1 second
> run mutation analysis : 2 seconds
=====
> Total : 5 seconds
=====
- Statistics
=====

```

```

>> Line Coverage: 19/23 (83%)
>> Generated 20 mutations Killed 14 (70%)
>> Mutations with no coverage 3. Test strength 82%
>> Ran 26 tests (1.3 tests per mutation)
Enhanced functionality available at https://www.arc4mutate.com/
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.097 s
[INFO] Finished at: 2025-08-29T12:54:01+05:30
[INFO] -----
PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3>
* History restored

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3> mvn clean test pitest:mutationCoverage
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.bankapp:simple-bank-app >-----
[INFO] Building simple-bank-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ simple-bank-app ---
[INFO] Deleting C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3\target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ simple-bank-app ---
[INFO] skip non existing resourceDirectory C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3\src\main\resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ simple-bank-app ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug target 17] to target\classes
[WARNING] location of system modules is not set in conjunction with -source 17
not setting the location of system modules may lead to class files that cannot run on JDK 17

```

```

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3> mvn clean test pitest:mutation
--release 17 is recommended instead of -source 17 -target 17 because it sets the location of system modules automatically
[INFO] --- resources:3.3.1:testResources (default-testResources) @ simple-bank-app ---
[INFO] skip non existing resourceDirectory C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3\src\test\resources
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ simple-bank-app ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug target 17] to target\test-classes
[WARNING] location of system modules is not set in conjunction with -source 17
not setting the location of system modules may lead to class files that cannot run on JDK 17
--release 17 is recommended instead of -source 17 -target 17 because it sets the location of system modules automatically
[INFO] --- surefire:3.2.5:test (default-test) @ simple-bank-app ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.bankapp.AccountTest
Insufficient funds. Current balance: 500.0
Deposit amount must be positive.
Withdrawal amount must be positive.
Deposit amount must be positive.
Withdrawal amount must be positive.
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.526 s -- in com.bankapp.AccountTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- pitest:1.7.4:mutationCoverage (default-cli) @ simple-bank-app ---

```

```

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3> mvn clean test pitest:mutation
[INFO] Found plugin : Default csv report plugin
[INFO] Found plugin : Default xml report plugin
[INFO] Found plugin : Default html report plugin
[INFO] Found plugin : Static initializer code detector plugin
[INFO] Found plugin : Excluded annotations plugin
[INFO] Found plugin : Try with resources filter
[INFO] Found plugin : Inlined finally block filter plugin
[INFO] Found plugin : Implicit null check filter
[INFO] Found plugin : Method reference null check filter
[INFO] Found plugin : For each loop filter
[INFO] Found plugin : Enum constructor filter
[INFO] Found plugin : Record junk mutation filter
[INFO] Found plugin : Logging calls filter
[INFO] Found plugin : Infinite for loop filter
[INFO] Found plugin : Long running iterator loop filter
[INFO] Found plugin : For loop counter filter
[INFO] Found plugin : Kotlin junk mutations filter
[INFO] Found plugin : Max mutations per class limit
[INFO] Found plugin : Equals shortcut equivalent mutant filter
[INFO] Found plugin : Trivial return vals equivalence filter
[INFO] Found plugin : Mutant export plugin
[INFO] Found shared classpath plugin : Default mutation engine
[INFO] Found shared classpath plugin : JUnit 5 test framework support
[INFO] Found shared classpath plugin : JUnit plugin
[INFO] Found shared classpath plugin : TestNG plugin
[INFO] Available mutators : EXPERIMENTAL_ARGUMENT_PROPAGATION, FALSE_RETURNS, TRUE_RETURNS, CONDITIONALS_BOUNDARY, CONSTRUCTOR_CALLS, EMPTY_RETURNS, INCREMENTS, INLINE_CO
NSTS, INVERT_NEGS, MATH, NEGATE_CONDITIONALS, NON_VOID_METHOD_CALLS, NULL_RETURNS, PRIMITIVE_RETURNS, REMOVE_CONDITIONALS_EQUAL_IF, REMOVE_CONDITIONALS_EQUAL_ELSE, REMOVE_C
ONDITIONALS_ORDER_IF, REMOVE_CONDITIONALS_ORDER_ELSE, RETURN_VALS, VOID_METHOD_CALLS, EXPERIMENTAL_BIG_DECIMAL, EXPERIMENTAL_BIG_INTEGER, EXPERIMENTAL_MEMBER_VARIABLE, EX
PERIMENTAL_NAKED_RECEIVER, REMOVE_INCREMENTS, EXPERIMENTAL_RETURN_VALUES_MUTATOR, EXPERIMENTAL_SWITCH, EXPERIMENTAL_BIG_DECIMAL, EXPERIMENTAL_BIG_INTEGER, ABS, AOD1, AOD2,
AOR1, AOR2, AOR3, AOR4, CRCR1, CRCR2, CRCR3, CRCR4, CRCR5, CRCR6, OBBN1, OBBN2, OBBN3, ROR1, ROR2, ROR3, ROR4, ROR5, UOI1, UOI2, UOI3, UOI4
[INFO] Adding org.pitest:pitest-junit5-plugin to SUT classpath
[INFO] Adding org.pitest:pitest to SUT classpath

```

```

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3> mvn clean test pitest:mutation
[INFO] Mutating from C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3\target\classes
2:40:24pm PIT >> INFO : Verbose logging is disabled. If you encounter a problem, please enable it before reporting an issue.
2:40:25pm PIT >> INFO : Incremental analysis reduced number of mutations by 0
2:40:25pm PIT >> INFO : Created 1 mutation test units in pre scan
2:40:26pm PIT >> INFO : Sending 1 test classes to minion
2:40:26pm PIT >> INFO : Sent tests to minion
2:40:29pm PIT >> INFO : MINION : Insufficient funds. Current balance: 500.0
2:40:29pm PIT >> INFO : MINION : Deposit amount must be positive.
2:40:29pm PIT >> INFO : MINION : Withdrawal amount must be positive.
2:40:29pm PIT >> INFO : MINION : Deposit amount must be positive.
2:40:29pm PIT >> INFO : MINION : Withdrawal amount must be positive.
2:40:30pm PIT >> INFO : MINION : Deposit amount must be positive.
2:40:30pm PIT >> INFO : MINION : Deposit amount must be positive.
2:40:30pm PIT >> INFO : MINION : Insufficient funds. Current balance: 500.0
2:40:30pm PIT >> INFO : MINION : Withdrawal amount must be positive.
2:40:30pm PIT >> INFO : MINION : Withdrawal amount must be positive.
|2:40:32pm PIT >> INFO : Calculated coverage in 6 seconds.
2:40:32pm PIT >> INFO : Incremental analysis reduced number of mutations by 0
2:40:32pm PIT >> INFO : Created 1 mutation test units
/2:40:43pm PIT >> INFO : Completed in 18 seconds
=====
- Mutators
=====
> org.pitest.mutationtest.engine.gregor.mutators.ConditionalsBoundaryMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
=====
> org.pitest.mutationtest.engine.gregor.mutators.returns.PrimitiveReturnsMutator
>> Generated 1 Killed 1 (100%)
> KILLED 1 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0

```

```

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3> mvn clean test pitest:mutation
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
-----
> org.pitest.mutationtest.engine.gregor.mutators.VoidMethodCallMutator
>> Generated 3 Killed 0 (0%)
> KILLED 0 SURVIVED 3 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
-----
> org.pitest.mutationtest.engine.gregor.mutators.returns.BooleanTrueReturnValsMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
-----
> org.pitest.mutationtest.engine.gregor.mutators.MathMutator
>> Generated 2 Killed 2 (100%)
> KILLED 2 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
-----
> org.pitest.mutationtest.engine.gregor.mutators.returns.BooleanFalseReturnValsMutator
>> Generated 2 Killed 2 (100%)
> KILLED 2 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
-----
> org.pitest.mutationtest.engine.gregor.mutators.returns.EmptyObjectReturnValsMutator
>> Generated 3 Killed 0 (0%)
> KILLED 0 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 3

```

```

PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3> mvn clean test pitest:mutation
> org.pitest.mutationtest.engine.gregor.mutators.NegateConditionalsMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
-----
- Timings
=====
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : 6 seconds
> build mutation tests : < 1 second
> run mutation analysis : 11 seconds
-----
> Total : 18 seconds
-----
- Statistics
=====
>> Line Coverage: 19/23 (83%)
>> Generated 20 mutations Killed 14 (70%)
>> Mutations with no coverage 3. Test strength 82%
>> Ran 25 tests (1.25 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 45.470 s
[INFO] Finished at: 2025-09-30T14:40:43+05:30
[INFO] -----
PS C:\Users\Dell\Downloads\VII SEM\STQA\LAB\EXP-3>

```

```

%STSTREE3,testWithdrawInsufficientFunds(com.bankapp.AccountTest),false,1,false,2,testWithdrawInsufficientFunds(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testWithdrawInsufficientFunds()]
%STSTREE4,testSuccessfulDeposit(com.bankapp.AccountTest),false,1,false,2,testSuccessfulDeposit(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testSuccessfulDeposit()]
%STSTREE5,testDepositZeroAmount(com.bankapp.AccountTest),false,1,false,2,testDepositZeroAmount(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testDepositZeroAmount()]
%STSTREE6,testSuccessfulWithdrawal(com.bankapp.AccountTest),false,1,false,2,testSuccessfulWithdrawal(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testSuccessfulWithdrawal()]
%STSTREE7,testWithdrawWithNegativeAmount(com.bankapp.AccountTest),false,1,false,2,testWithdrawWithNegativeAmount(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testWithdrawWithNegativeAmount()]
%STSTREE8,testWithdrawExactBalance(com.bankapp.AccountTest),false,1,false,2,testWithdrawExactBalance(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testWithdrawExactBalance()]
%STSTREE9,testDepositWithNegativeAmount(com.bankapp.AccountTest),false,1,false,2,testDepositWithNegativeAmount(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testDepositWithNegativeAmount()]
%STSTREE10,testWithdrawZeroAmount(com.bankapp.AccountTest),false,1,false,2,testWithdrawZeroAmount(),,[engine:junit-jupiter]/[class:com.bankapp.AccountTest]/[method:testWithdrawZeroAmount()]
%TESTS 3,testWithdrawInsufficientFunds(com.bankapp.AccountTest)
%TESTE 3,testWithdrawInsufficientFunds(com.bankapp.AccountTest)
%TESTS 4,testSuccessfulDeposit(com.bankapp.AccountTest)
%TESTE 4,testSuccessfulDeposit(com.bankapp.AccountTest)
%TESTS 5,testDepositZeroAmount(com.bankapp.AccountTest)
%TESTE 5,testDepositZeroAmount(com.bankapp.AccountTest)
%TESTS 6,testSuccessfulWithdrawal(com.bankapp.AccountTest)
%TESTE 6,testSuccessfulWithdrawal(com.bankapp.AccountTest)
%TESTS 7,testWithdrawWithNegativeAmount(com.bankapp.AccountTest)
%TESTE 7,testWithdrawWithNegativeAmount(com.bankapp.AccountTest)
%TESTS 8,testWithdrawExactBalance(com.bankapp.AccountTest)
%TESTE 8,testWithdrawExactBalance(com.bankapp.AccountTest)
%TESTS 9,testDepositWithNegativeAmount(com.bankapp.AccountTest)
%TESTE 9,testDepositWithNegativeAmount(com.bankapp.AccountTest)
%TESTS 10,testWithdrawZeroAmount(com.bankapp.AccountTest)
%TESTE 10,testWithdrawZeroAmount(com.bankapp.AccountTest)
%RUNTIME242

```

Test Runner for Java

- testDepositWithNegativeAmount() \$symbol-cla...
- testDepositZeroAmount() \$symbol-cla...
- testSuccessfulDeposit() \$symbol-cla...
- testSuccessfulWithdrawal() \$symbol-cla...
- testWithdrawExactBalance() \$symbol-cla...
- testWithdrawInsufficientFunds() \$symbol-cla...
- testWithdrawWithNegativeAmount() \$symbol-cla...
- testWithdrawZeroAmount() \$symbol-cla...

> 1 older result

Questions:**1. What is the equivalent Mutant? Explain with examples.**

Ans: In mutation testing, a mutant is a modified version of the original program, created by making small syntactic changes to the code, such as altering operators, changing variables, or modifying expressions. The idea is to introduce faults in the code (mutants) and test whether the existing test cases can detect these faults. The mutant that behaves differently from the original program is said to be "killed" by the test case, meaning that the test case has successfully detected the introduced fault. If the mutant produces the same output as the original program, it is considered "alive," meaning the test case failed to detect the error.

Examples of Mutants:

1. Value Mutation:

- Original Program: `if (x > 5) { System.out.println("Greater"); }`
- Mutant: `if (x < 5) { System.out.println("Greater"); }`
- Explanation: The condition is mutated by changing the `>` operator to `<`.

2. Statement Mutation:

- Original Program: `return x + y;`
- Mutant: `return x - y;`
- Explanation: The arithmetic operation `+` is replaced with `-` to check for potential errors in the program's logic.

3. Logical Mutation:

- Original Program: `if (a && b) { // do something }`
- Mutant: `if (a || b) { // do something }`
- Explanation: The logical AND (`&&`) is replaced with a logical OR (`||`) to test the program's handling of different logical conditions.

Mutants are a way to measure the effectiveness of test cases by evaluating whether they can catch these small changes. If a test case is unable to detect the mutation, the test case is considered inadequate, and additional tests may be needed to achieve full coverage.

Outcomes: CO2 — Demonstrate designing and execution of test cases using testing techniques.

Conclusion: (Conclusion to be based on outcomes)

Mutation testing is a powerful technique for assessing the adequacy of test cases by introducing small changes (mutants) to the code. The effectiveness of the test cases is measured by their ability to detect these faults, with a 100% mutation score indicating that the test suite is fully adequate. Tools like PITest automate mutation testing, making it more efficient and cost-effective. This experiment demonstrates how mutation testing helps in improving the quality of the test cases and the overall software development process.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

Reference Websites:

1. Mutation Tasting Tutorial
https://www.tutorialspoint.com/software_testing_dictionary/mutation_testing.htm
<https://www.guru99.com/mutation-testing.html>
 2. Mutation Testing
<https://www.youtube.com/watch?v=1DvJeMmKrUE>
 3. Mutation Testing With PIT
<https://www.youtube.com/watch?v=3A1tHE8z64>
-

